1. Expressions

An **expression** is a combination of operators, constants, and variables that are evaluated to produce a single value.

- **Definition:** The smallest unit of computation that returns a result.
- **Key Points:**
  - Every expression has a type (e.g., integer, boolean).
  - Expressions can be as simple as `5` or as complex as a mathematical formula.
  - They end with a semicolon `;` to become an **expression statement**.

Example Code:

C++
```
int x = 10;
int y = 20;
int sum = x + y; // 'x + y' is an expression
bool isGreater = (x > y); // '(x > y)' is a boolean expression
```

---

2. Functions

A **function** is a self-contained block of code that performs a specific task. It allows you to reuse code instead of writing it multiple times.

- **Definition:** A named module of code that can be "called" from other parts of the program.
- **Key Points:**
  - **Declaration:** Tells the compiler about the function's name and return type.
  - **Definition:** The actual body of the function.
  - **Parameters:** Values passed into the function.
  - **Return Value:** The result the function sends back.

Example Code:

C++
```
#include <iostream>
using namespace std;

// Function Definition
int multiply(int a, int b) {
    return a * b;
}

int main() {
    int result = multiply(5, 4); // Function Call
    cout << "Product: " << result;
    return 0;
```

```
}
```

---

3. Inline Functions

An **inline function** is a request to the compiler to replace the function call with the actual code of the function to save time.

- **Definition:** A function qualified with the `inline` keyword.
- **Key Points:**
  - Used for **small, frequently called functions**.
  - **Efficiency:** It eliminates the "overhead" of jumping to a function and back (saving memory stack operations).
  - **Compiler Choice:** The compiler may ignore the `inline` request if the function is too complex (like if it contains loops).

Example Code:

C++
```
inline int square(int s) {
    return s * s;
}

int main() {
    cout << "Square of 5: " << square(5);
    // Compiler replaces 'square(5)' with '5 * 5' during compilation
    return 0;
}
```

---

4. Recursion

Recursion is a technique where a function calls itself to solve a smaller version of the same problem.

- **Definition:** A process in which a function calls itself directly or indirectly.
- **Key Points:**
  - **Base Case:** The condition that stops the recursion (prevents infinite loops).
  - **Recursive Step:** The part where the function calls itself with a modified argument.
  - **Memory:** Each call creates a new layer on the "stack."

Example Code (Calculating Factorial):

C++
```
#include <iostream>
using namespace std;
```

```
int factorial(int n) {
    // Base Case
    if (n <= 1)
        return 1;
    // Recursive Step
    else
        return n * factorial(n - 1);
}

int main() {
    int num = 5;
    cout << "Factorial of " << num << " is " << factorial(num);
    return 0;
}
```

Comparison Summary

Feature	Function	Inline Function	Recursion
Purpose	Code Reusability	Speed Optimization	Solving complex logic simply
Code Size	No change	Increases binary size	Small code, high memory use
Calling	Standard call	Code is copied in-place	Calls itself